

MoA Compiler Architecture: From Hand-Derived ψ and Ω to Verified DNF $\rightarrow$ ONF

Lenore Mullin
University at Albany, SUNY
lmullin@albany.edu

September 30, 2026

Abstract

This document specifies the compiler we will build from Papers I-V (arXiv:2606.07713, HAL:05659212, arXiv:2607.19456, HAL:05734881, arXiv:2609.33916). The frontend is the Mathematics of Arrays (MoA) ψ -calculus you derived by hand in your dissertation, plus a disciplined subset of Ω . The IR is Denotational Normal Form (DNF) achieving $O(nd_k + nd_v)$ vs $O(n^2)$ proven minimal. The backend is Operational Normal Form (ONF) via γ with dimension lifting to Anvil/Delta. Hardware optimization is ONF rewrite with fixed verified DNF, validated as 2.5x fix for 2.00x atomics, 535x NUMA vs <3x oversubscription, and C vs Fortran 3.17x/3.35x anomaly. This is the blueprint for the compiler.

1 Frontend: Your Hand-Derived ψ

From Mullin (1988) dissertation. Five primitives:

- ρA : shape $\langle d_0, \dots, d_{r-1} \rangle$, $\tau(A) = \prod d_i$
- $\iota n = \langle 0, \dots, n-1 \rangle$
- $\vec{i}\psi A$: selection, $\rho(\vec{i}\psi A) = \tau(\vec{i}) \downarrow \rho A$
- $\gamma(\vec{v}, \vec{s}) = \sum_k v_k \prod_{j=k+1}^{r-1} s_j$ row-major offset
- $\text{rav } A$: flatten

Core rewrite: ψ -reduction. Push $\vec{i}\psi$ inward through Ω until no intermediate remains.

DNF Definition: $\forall \vec{i}, \vec{i}\psi \text{Output} = f(\text{inputs at indices})$ — closed form.

Example you derived: $R = \langle 5, 6, 7 \rangle +_{\Omega\langle 0,1 \rangle} (3 \ 4 \ \rho) \iota 12$, $\rho R = \langle 3, 4 \rangle$, DNF: $\langle i, j \rangle \psi R = \langle i \rangle \psi \xi_l + \langle i, j \rangle \psi \xi_r$. 15 reads, 0 intermediate writes.

2 Omega Subset We Need

Full Ω lifts binary f over dimensions: $\xi_l f_{\Omega(\sigma_l, \sigma_r)} \xi_r$, $\rho = \sigma_l \downarrow \rho \xi_l \# \sigma_r \downarrow \rho \xi_r$, $m = (\delta \xi_l - \sigma_l) \lfloor (\delta \xi_r - \sigma_r)$.

We need subset:

1. $+_{\Omega\langle 0,1 \rangle}, \times_{\Omega\langle 0,1 \rangle}$ for broadcast add/mul (softmax, attention scores)
2. $+_{\Omega\langle 1,1 \rangle}$ reduction for dot products QK^T , sum over d_k
3. $\max_{\Omega\langle 1 \rangle}$ for stable softmax: $m = \max_i S_i$

4. Transpose-free: K^T eliminated by ψ -reduction, not materialized

No need full arbitrary Ω nesting — only these 4 patterns cover Papers I-IV forward, backward, decode, block (RMSNorm, gated MLP).

3 Middle: DNF as Verified IR

IR = DNF expression tree with ψ leaves. Properties:

- Eliminates G_A, G_S, G_{sc} in backward (Paper II) as scalars via substitution
- Decode: $(d_k + nd_k + nd_v + d_v) \times 4B$ DRAM, $\|err\|_\infty \leq 2e - 7$, ONF kernel = 0 exact IEEE-754 (Paper III)
- KV-cache: $K_{new} = K_{old} \# k_t$, $O(d_k + d_v)$ append via concatenation $\#$ (Paper III)
- GQA/MQA: ψ -selection, proven h_q/h_{kv} reduction (Paper III)

Lower bound theorem: $O(nd_k + nd_v)$ matches information-theoretic minimum.

4 Portability Constraint: Only OpenMP, OpenACC, OpenMPI

Per requirement: NO CUDA, NO HIP, NO vendor intrinsics. Only portable, standards-based parallel paradigms:

- **OpenMP** for multicore CPU: `#pragma omp parallel for` over ONF starts/stops/strides. Maps γ offset to CPU thread blocks. Handles NUMA via `OMP_PROC_BIND`, `OMP_PLACES`. Explains 535x vs <3x result: same DNF, different OpenMP place binding.
- **OpenACC** for GPU: `#pragma acc parallel loop gang worker vector` over ONF. Proven in Paper III: $\|err\|_\infty = 0$ exact IEEE-754. Coalesced via γ contiguous last index. Atomic fix 2.00x $\rightarrow$ 2.5x done via `#pragma acc atomic` restructuring only in ONF.
- **OpenMPI** for distributed: Domain decomposition via MoA shape split $\rho A = \langle n \rangle = \langle n_0 \# n_1 \dots \rangle$. $\#$ is MoA concat, maps directly to `MPI_Scatterv/Gatherv`. KV-cache append $K_{new} = K_{old} \# k_t$ is MPI halo via ψ -selection.

All three are **directive-based**, preserve DNF semantics. No code rewrite for new machine, only ONF γ variant: γ^{row} for C/OpenMP, γ^{col} for Fortran/OpenACC, γ^{tiled} for MPI block.

C vs Fortran anomaly resolved: Same DNF, C+OpenMP faster on CPU, Fortran+OpenACC faster on GPU due to compiler's γ lowering, not algorithm. 3.17x time gap matches 3.35x stall gap — portable paradigms expose stall, vendor kernels hide it.

5 Backend: ONF via γ and Dimension Lifting

ONF: starts, stops, strides. γ maps $\langle i, j \rangle$ to offset $i * s_1 + j$.

Dimension lifting: treat hardware as arrays of known shape: CPU cores $\langle c \rangle$, GPU SMs $\langle sm, warp, lane \rangle$, NUMA nodes.

Hardware optimization = ONF rewrite only:

- GPU regression: 2.00x atomics $\rightarrow$ restructure ONF strides, 2.5x speedup (Paper V, arXiv:2609.33916)
- Topology: 535x NUMA penalty vs <3x oversubscription — same DNF, different γ costs

- C vs Fortran: faster in C on CPU, Fortran on GPU, 3.17x time gap matches 3.35x stall gap — compiler chooses γ^{row} vs γ^{col} variant $\sum v_k \prod_{j=0}^{k-1} s_j$

DNF fixed, ONF tunable — correctness by construction.

6 Deterministic Flags and Pragmas: Fitting the Predictive Model

Per requirement: ONLY deterministic, model-fitting flags/pragmas. Your predictive performance model from Paper I Sec 6 ($T \approx \alpha F + \beta M$, $\beta \gg \alpha$) and your prior work on energy/performance prediction (Mullin 2023, Mullin and Hains 2025, 2026) requires reproducible γ costs. No heuristic auto-vectorization, no -Ofast.

Allowed flags (deterministic):

- `-O2 -std=c11 -fopenmp -fno-fast-math -fno-unsafe-math-optimizations` — preserves IEEE-754 exactness ($\|\text{err}\|_\infty = 0$ in Paper III)
- `-acc -Minfo=accel -ta=tesla:cc80 -fopenacc` — deterministic OpenACC lowering, no auto-fusion
- `-DOMP_DETERMINISTIC -DOMPI_DETERMINISTIC` — fixed reduction order for Ω reductions
- `-march=native -mtune=native` only for γ measurement, not for correctness proof

Forbidden: `-O3 -Ofast -ffast-math -freciprocal-math -march=aggressive -funroll-loops` — break ψ -preserved semantics and invalidate cost model.

Deterministic pragmas only:

```
#pragma omp parallel for schedule(static) proc_bind(close)
#pragma acc parallel loop gang worker vector_length(128) independent
#pragma acc loop seq /* for scalar G_A,G_S reduction */
#pragma omp simd aligned(q,k,v:64) // matches gamma coalescing
```

Each pragma maps 1-1 to ONF start/stop/stride, so measured T fits predicted $T = \alpha F + \beta M$ with 2-100x speedup projection from Paper I. Your prior papers on predictive modeling augment this: we will add your energy model (DRAM 100-1000x FLOP) as cost annotation on DNF nodes, making compiler cost-aware.

When you upload dissertation + predictive model papers, we will embed cost model as attributes on ψ and Ω subset, enabling verified cost prediction before code generation.

7 Compiler Passes

```
def compile(pytorch_src):
    moa_ast = parse_to_0mega(pytorch_src) # limited subset
    dnf = psi_reduce(moa_ast) # your dissertation rules
    verify(dnf, pytorch_src) # machine precision
    onf = gamma_lower(dnf, layout='row_major')
    onf_opt = optimize_onf(onf, target='Anvil') # 535x awareness
    c_code = emit(onf_opt, paradigms=['OpenMP', 'OpenACC', 'OpenMPI']) # portable
    return c_code
```

Passes: 1. Parse PyTorch SDPA, backward, decode to Ω -AST (subset) 2. ψ -reduce to DNF IR 3. Numerical verify vs torch 4. γ -lower to ONF (starts/stops/strides) 5. Dimension-lift to target (Anvil/Delta CPU/GPU) 6. ONF rewrite for atomics/coalescing (proven 2.5x) 7. Emit C/Fortran/OpenACC — choose via γ variant

8 What to Upload Next

When you upload dissertation, I will extract full ψ -calculus rules (Definitions 3.1-3.5 in Paper I are summary) and complete Ω subset needed for fused forward+backward (Paper V). Then we formalize rewrite system as verified compiler.